

Seeders

We know how to create and use factories and we used them within Tinker. We can create seeders to populate our database with data right from the command line with Artisan. This is useful for testing and development purposes. We can also use seeders to populate our database with initial data when we deploy our application.

If you open the `database/seeds` directory, you will see a file called `DatabaseSeeder.php`. This is the default seeder that Laravel creates for us. Seeders only have a single method called `run`. This method is called when we run the `db:seed` command. This particular seeder uses the user factory to create a user.

Create a Random User Seeder

Let's create a user seeder that will fun our factory and create 10 random users on the fly.

Create the seeder with the following command:

```
php artisan make:seeder RandomUserSeeder
```

This will create a new file at `database/seeds/RandomUserSeeder.php`. This file will contain a class called `RandomUserSeeder` that extends the `Seeder` class. It will have a method called `run`. This is where we will define the logic for populating our database with data.

Bring in the `User` model:

```
use App\Models\User;
```

Add the following code to the `run` method:

```
public function run(): void
{
    // Create 10 users using the UserFactory
    $users = User::factory(10)->create();

    echo "Users created successfully!";
}
```

We can run the seeder with the following command:

```
```bash
php artisan db:seed --class=RandomUserSeeder
```

## Create a Random Job Seeder

Let's create a new seeder called `RandomJobSeeder.php` in the `database/seeder` directory. We can use the `make:seeder` command to create a new seeder.

```
php artisan make:seeder RandomJobSeeder
```

This will create a file at `database/seeder/RandomJobSeeder.php`. It will create a class called `RandomJobSeeder` that extends the `Seeder` class with a method called `run`. The `run` method is where we will define the logic for populating our database with data.

Let's bring in the `Job` model:

```
use App\Models\Job;
```

Now in the `run` method, we will use the `Job` model to create a new job listing. We will use the `factory` method to create a new job listing. We will use the `count` method to specify the number of job listings we want to create. We will use the `create` method to create the job listing.

```
public function run(): void
{
 // Generate 10 job listings using the factory
 Job::factory()->count(10)->create();

 echo "Jobs created successfully!";
}
```

This will create 10 job listings using the `JobFactory` that we created earlier. We can now run the seeder using the `db:seed` command:

```
php artisan db:seed --class=RandomJobSeeder
```

This will add 10 job listings to our database. With 10 new users.

If you want to keep it this way you can, but I would like to have a seeder that creates the same group of job listings. In the next lesson, we will have the seeder put in some hardcoded jobs and we will look at truncating the tables first and calling seeders from within another using the `call` method.